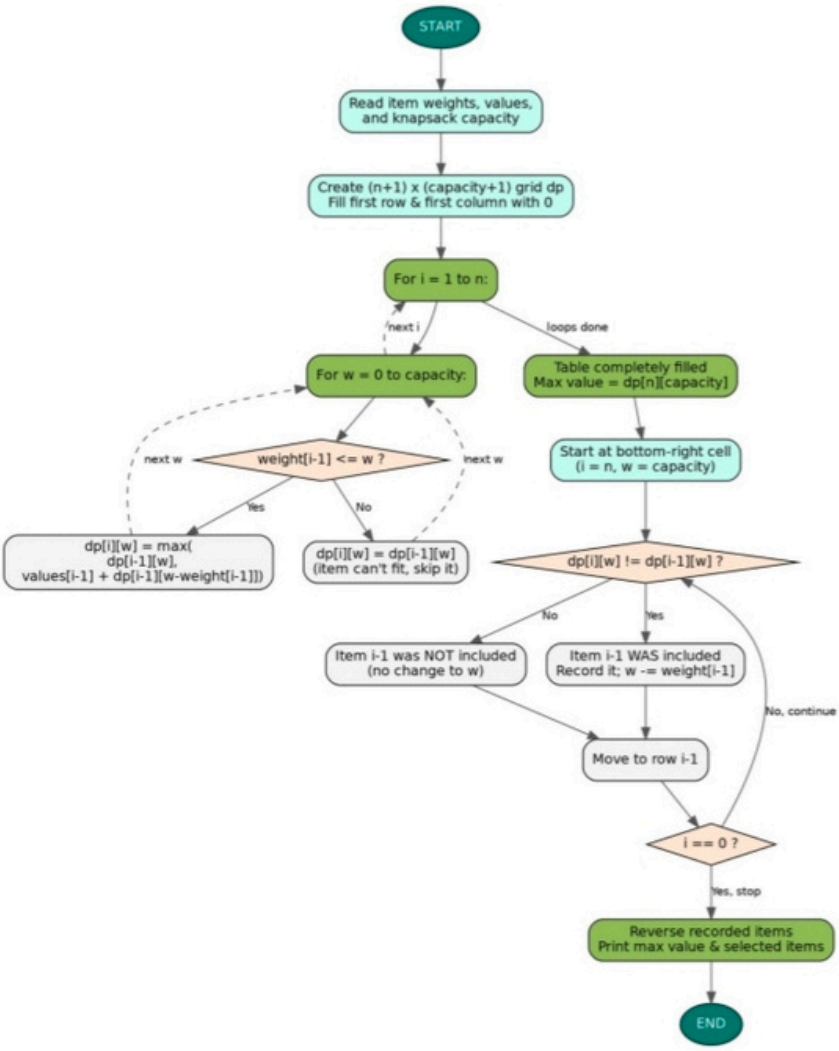
Algorithm: Optimal Selection of Items Within a Weight Constraint

(The Classic 0/1 Knapsack Problem)

1. Start the program.
2. Define the item weights, item values, and the maximum knapsack capacity.
3. Implement a brute-force recursive method that tries including or excluding each item.
4. Implement a top-down dynamic programming method using memoization to remember solved states.
5. Implement a bottom-up dynamic programming method using a table.
6. Create a DP table with rows representing items and columns representing capacities from 0 to the maximum capacity.
7. For each item and capacity, check whether the item can fit.
8. If the item cannot fit, copy the value from the previous row.
9. Otherwise, calculate both choices: skip the item or take the item.
10. Store the larger of the two values in the DP table.
11. Backtrack through the DP table to identify the selected item indices.
12. Display the maximum value and the selected items.
13. Compare the brute-force, top-down, and bottom-up results.
14. Perform a timing comparison between brute force and bottom-up approaches.
15. Display the timing results.
16. End the program.

Flowchart — 0/1 Knapsack Using Dynamic Programming

START



0/1 Knapsack Problem — Python Source Code

```
import timeit

# 1. BRUTE FORCE
def knapsack_brute_force(weights, values, capacity, n=None):
    if n is None:
        n = len(weights)

    ifn== 0 or capacity == 0:
        return 0

    ifweights[n - 1] > capacity:
        return knapsack_brute_force(
            weights, values, capacity, n - 1
        )

    include = values[n - 1] + knapsack_brute_force(
        weights, values, capacity - weights[n - 1], n - 1
    )
    exclude = knapsack_brute_force(
        weights, values, capacity, n - 1
    )

    return max(include, exclude)


# 2. TOP-DOWN (MEMOIZATION)
def knapsack_top_down(weights, values, capacity):
    n = len(weights)
    memo = {}

    def helper(i, remaining_capacity):
        ifi== 0 or remaining_capacity == 0:
            return 0

        if(i, remaining_capacity) in memo:
            return memo[(i, remaining_capacity)]

        ifweights[i-   1] > remaining_capacity:
            result =helper(i - 1, remaining_capacity)
        else:
            include=values[i - 1] + helper(
                i-1,   remaining_capacity - weights[i - 1]
            )
            exclude=helper(i - 1, remaining_capacity)
            result =max(include, exclude)

        memo[(i, remaining_capacity)] = result
        return result

    return helper(n, capacity)


# 3. BOTTOM-UP (TABULATION)
def build_knapsack_table(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        forwinrange(capacity + 1):
            if weights[i - 1] > w:
                dp[i][w]= dp[i - 1][w]
            else:
                skip= dp[i-1][w]
                take=  values[i - 1]  + dp[i - 1][
                      w-weights[i - 1]
                ]
                dp[i][w]= max(skip, take)

    return dp


def reconstruct_selected_items(dp, weights, values, capacity):
    n = len(weights)
    w = capacity
    selected = []

    for iinrange(n, 0, -1):
        ifdp[i][w] != dp[i - 1][w]:
            selected.append(i - 1)
```

```
            w-= weights[i - 1]

    selected.reverse()
    return selected


def knapsack_bottom_up(weights, values, capacity):
    dp = build_knapsack_table(weights, values, capacity)
    n = len(weights)
    max_value = dp[n][capacity]
    selected = reconstruct_selected_items(
        dp, weights, values, capacity
    )
    return max_value, selected, dp


# 4. DEMO / DRIVER CODE
if __name__ == "__main__":
weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
capacity = 5

    print("Items (weight, value):",
          list(zip(weights, values)))
    print(f"Knapsack capacity: {capacity}\n")

    print(f"[1] Brute force : "
          f"{knapsack_brute_force(weights, values, capacity)}")
    print(f"[2] Top-down    : "
          f"{knapsack_top_down(weights, values, capacity)}")

    max_value, selected, dp = knapsack_bottom_up(
        weights, values, capacity
    )
    print(f"[3] Bottom-up   : {max_value}")
    print(f"    Selected items (0-indexed): {selected}")

    for idx in selected:
        print(f"      item {idx}: "
              f"weight={weights[idx]}, value={values[idx]}")

    assertknapsack_brute_force(
        weights, values, capacity
    )==max_value
    assert knapsack_top_down(
        weights, values, capacity
    )==max_value

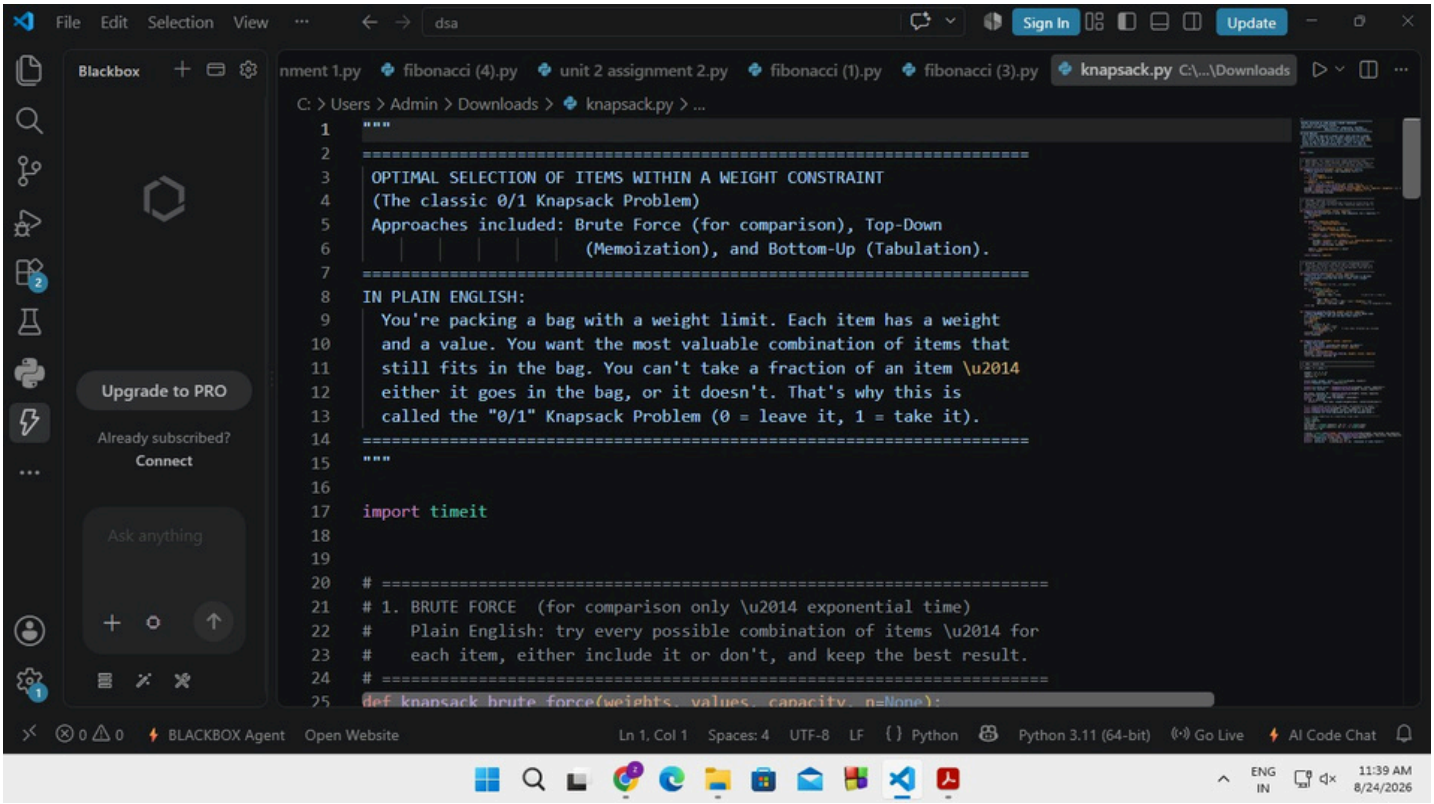
    print("\n[check] All three approaches agree "
          "on the maximum value.")

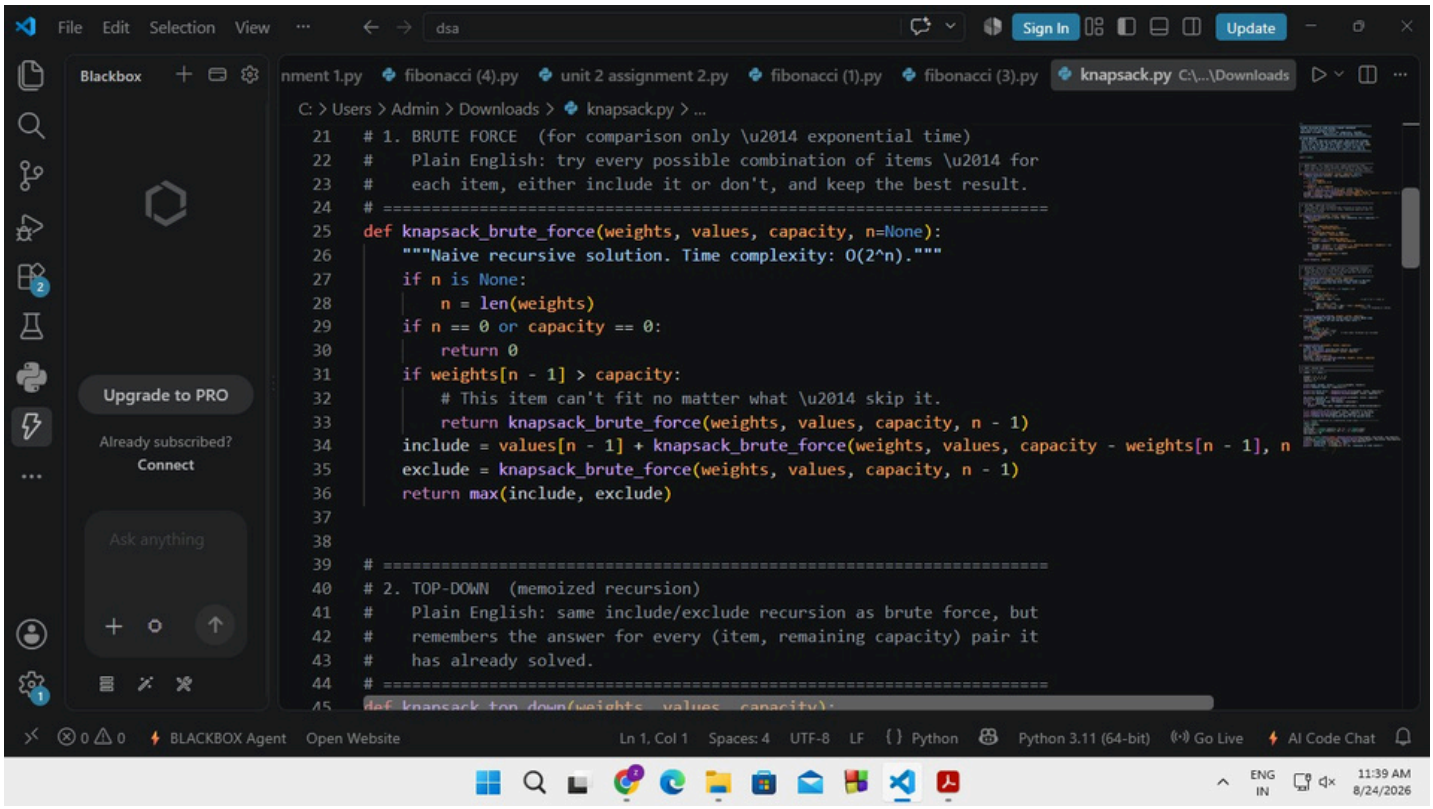
    import random
    random.seed(42)

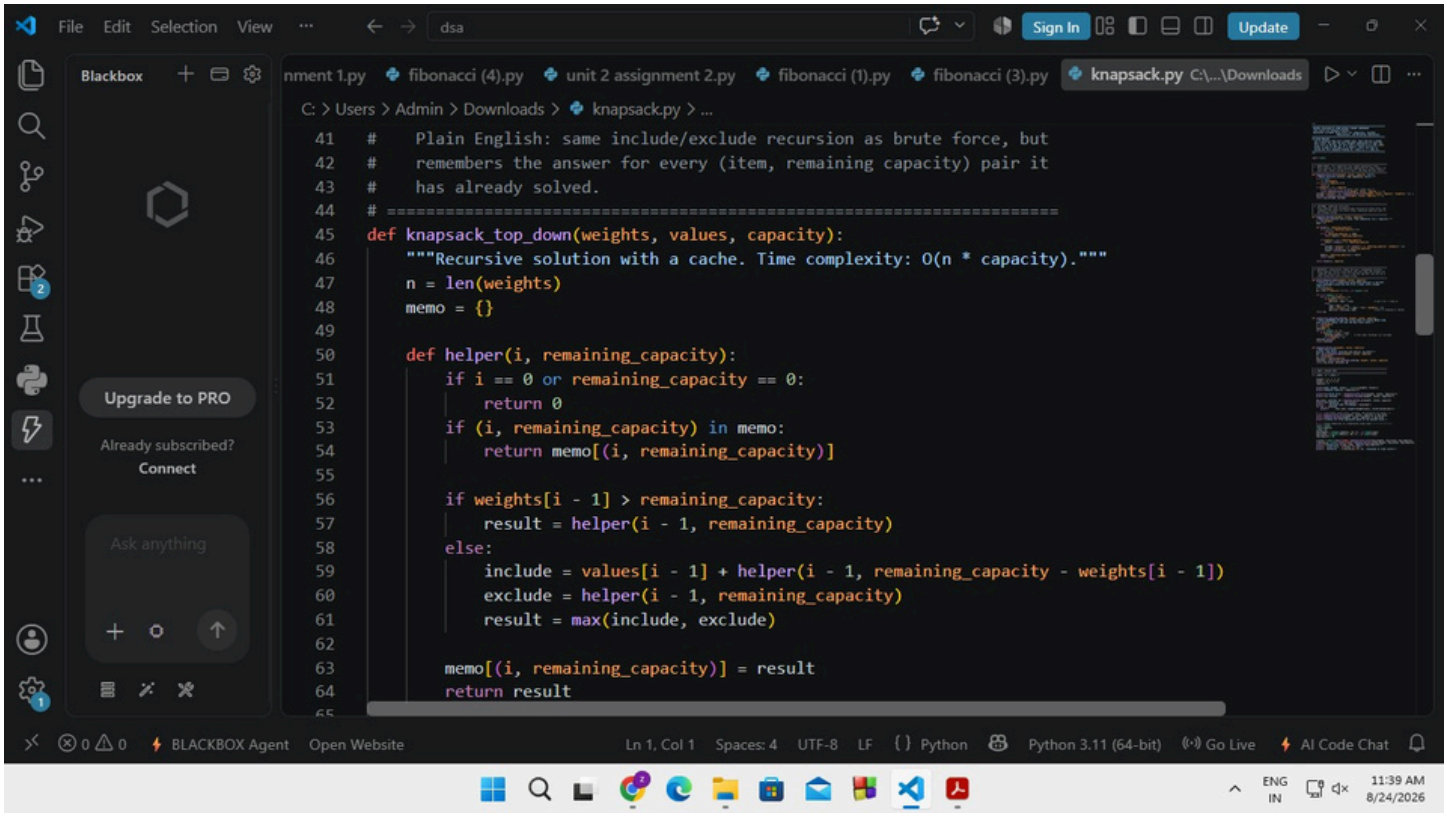
    n_big = 22
    big_weights=[random.randint(1, 15)
                   for _ in range(n_big)]
    big_values =[random.randint(1, 20)
                  for _ in range(n_big)]
    big_capacity = 50

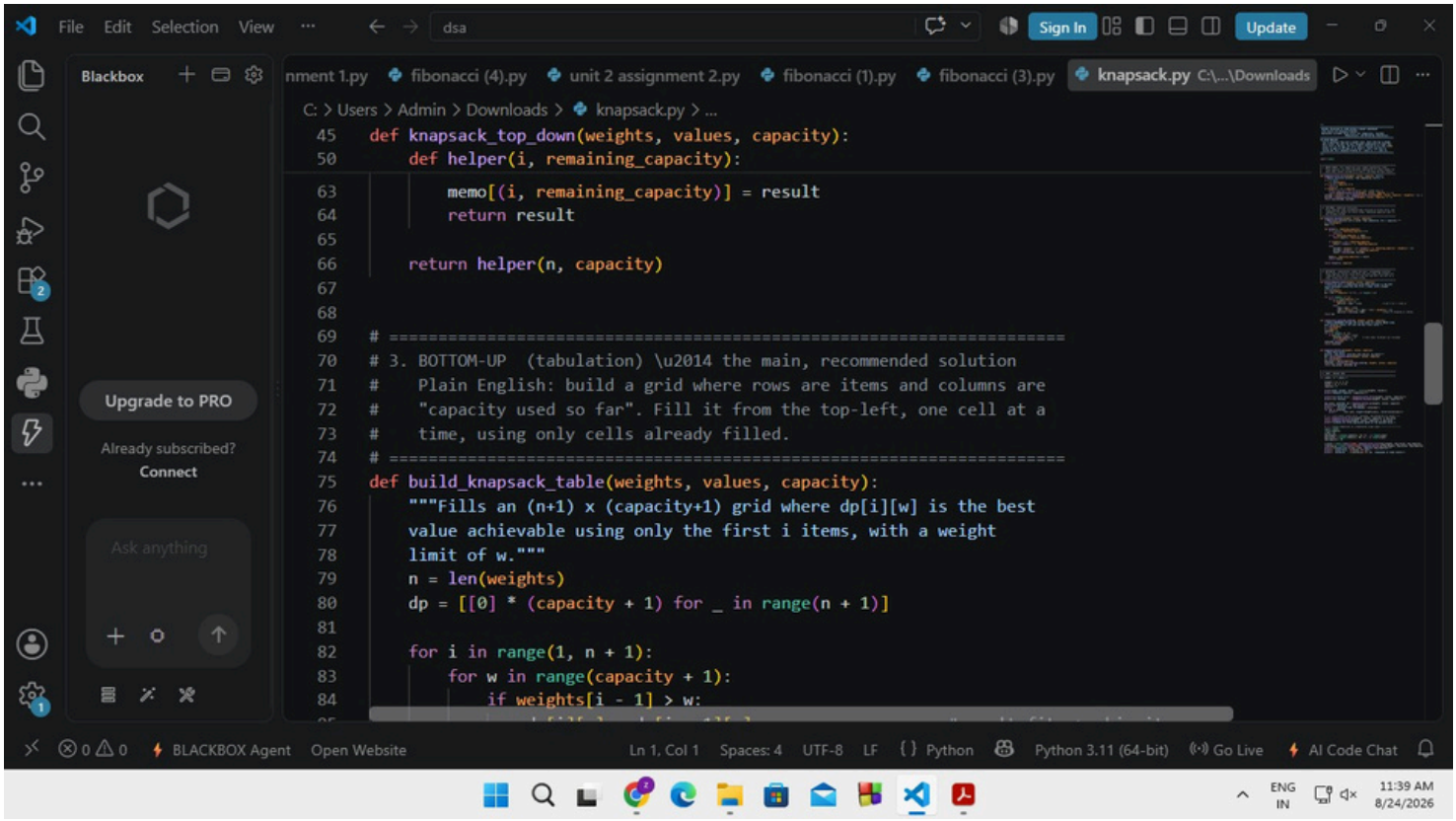
    t_brute=timeit.timeit(
        lambda:knapsack_brute_force(
            big_weights, big_values, big_capacity
        ), number=1
    )
    t_bottom_up=timeit.timeit(
        lambda:knapsack_bottom_up(
            big_weights, big_values, big_capacity
        ), number=1
    )

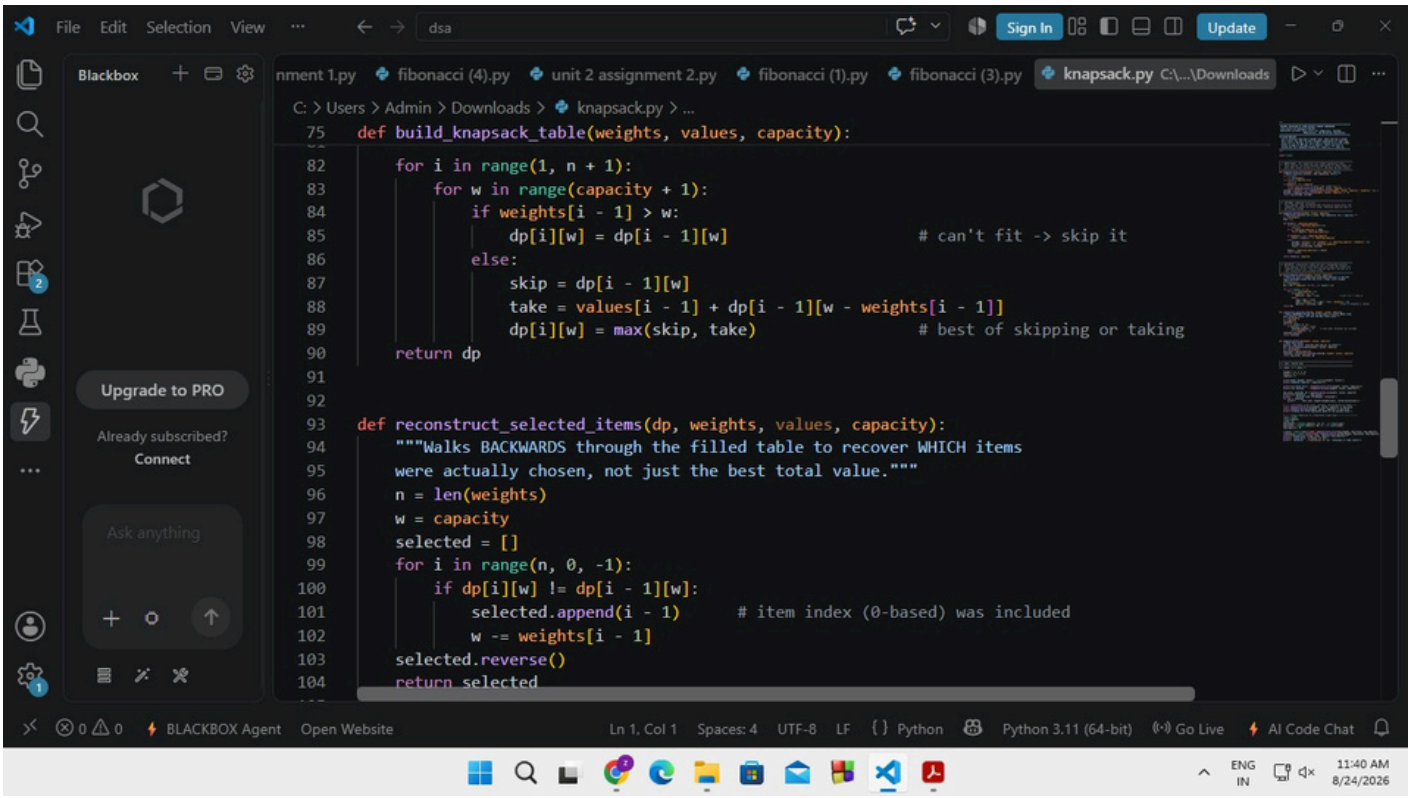
    print(f"\nTiming with {n_big} items, "
          f"capacity {big_capacity}:")
    print(f" brute force : {t_brute:.5f} sec")
    print(f" bottom-up    : {t_bottom_up:.8f} sec")
```

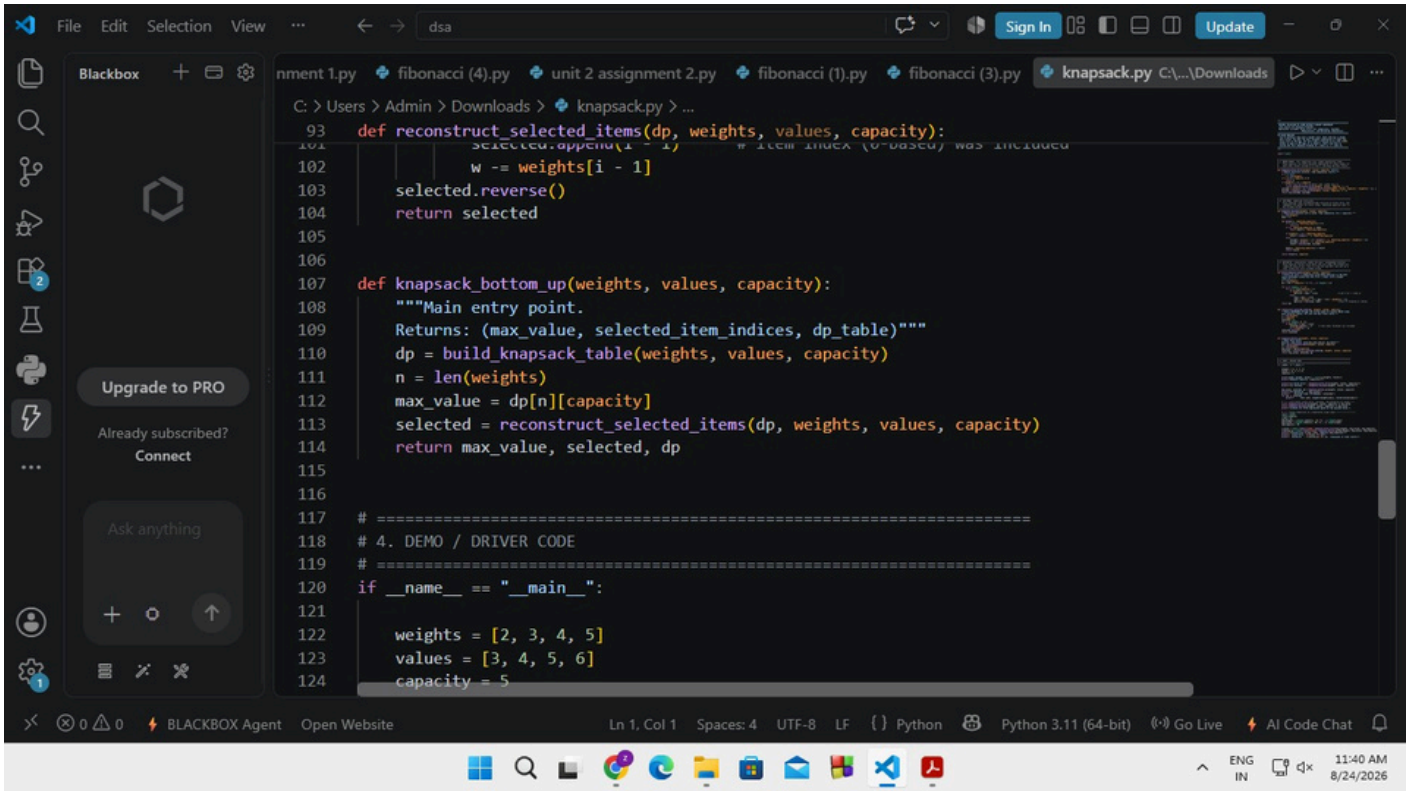

File Edit Selection View
dsa
Sign In
Update
Blackbox
nment 1.py
fibonacci (4).py
unit 2 assignment 2.py
fibonacci (1).py
fibonacci (3).py
knapsack.py C:\...\Downloads
C: > Users > Admin > Downloads > knapsack.py > ...
"""
OPTIMAL SELECTION OF ITEMS WITHIN A WEIGHT CONSTRAINT
(The classic 0/1 Knapsack Problem)
Approaches included: Brute Force (for comparison), Top-Down
(Memoization), and Bottom-Up (Tabulation).
IN PLAIN ENGLISH:
You're packing a bag with a weight limit. Each item has a weight
and a value. You want the most valuable combination of items that
still fits in the bag. You can't take a fraction of an item \u2014
either it goes in the bag, or it doesn't. That's why this is
called the "0/1" Knapsack Problem (0 = leave it, 1 = take it).
"""
import timeit
1. BRUTE FORCE (for comparison only \u2014 exponential time)
Plain English: try every possible combination of items \u2014 for
each item, either include it or don't, and keep the best result.
def knapsack_brute_force(weights, values, capacity, n=None):
Upgrade to PRO
Already subscribed?
Connect
Ask anything
BLACKBOX Agent
Open Website
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.11 (64-bit)
Go Live
AI Code Chat
ENG
IN
11:39 AM
8/24/2026

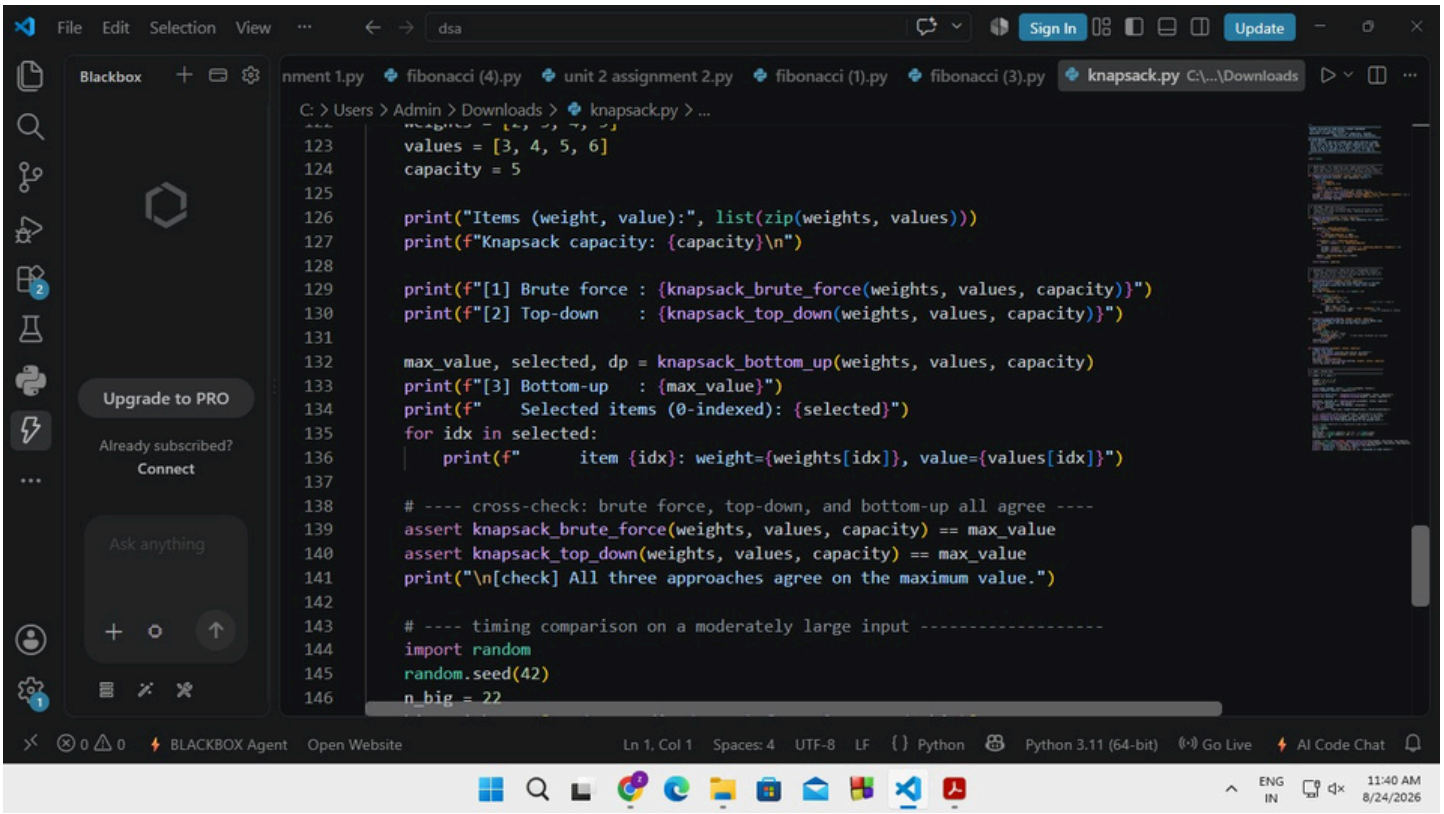

File Edit Selection View
dsa
Sign In
Update
Blackbox
nment 1.py
fibonacci (4).py
unit 2 assignment 2.py
fibonacci (1).py
fibonacci (3).py
knapsack.py C:\...\Downloads
C: > Users > Admin > Downloads > knapsack.py > ...
1. BRUTE FORCE (for comparison only \u2014 exponential time)
Plain English: try every possible combination of items \u2014 for
each item, either include it or don't, and keep the best result.
def knapsack_brute_force(weights, values, capacity, n=None):
"""Naive recursive solution. Time complexity: O(2^n)."""
if n is None:
n = len(weights)
if n == 0 or capacity == 0:
return 0
if weights[n - 1] > capacity:
This item can't fit no matter what \u2014 skip it.
return knapsack_brute_force(weights, values, capacity, n - 1)
include = values[n - 1] + knapsack_brute_force(weights, values, capacity - weights[n - 1], n
exclude = knapsack_brute_force(weights, values, capacity, n - 1)
return max(include, exclude)
2. TOP-DOWN (memoized recursion)
Plain English: same include/exclude recursion as brute force, but
remembers the answer for every (item, remaining capacity) pair it
has already solved.
Upgrade to PRO
Already subscribed?
Connect
Ask anything
BLACKBOX Agent
Open Website
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.11 (64-bit)
Go Live
AI Code Chat
ENG
IN
11:39 AM
8/24/2026

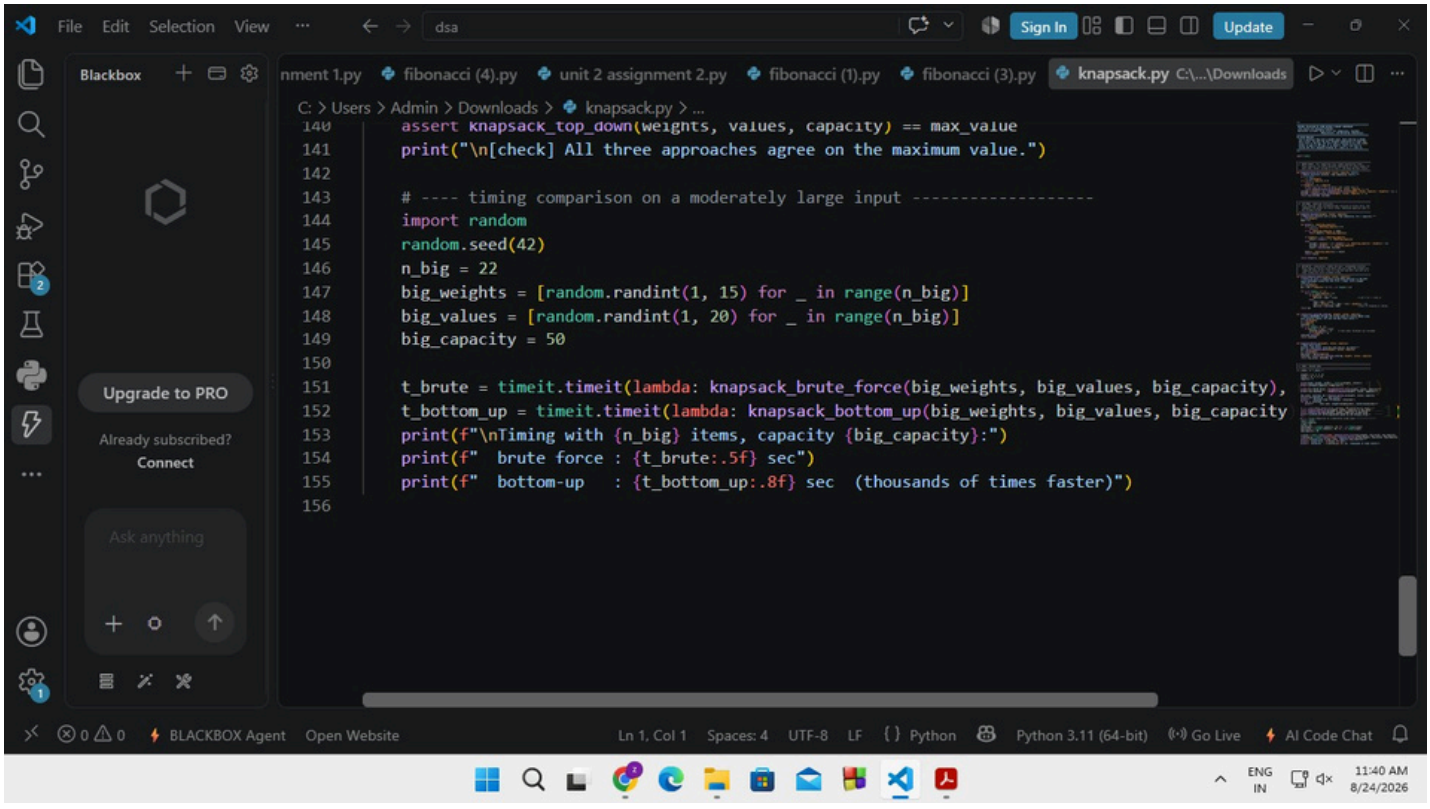

File Edit Selection View
dsa
Sign In
Update
Blackbox
nment 1.py
fibonacci (4).py
unit 2 assignment 2.py
fibonacci (1).py
fibonacci (3).py
knapsack.py C:\...\Downloads
C: > Users > Admin > Downloads > knapsack.py > ...
Upgrade to PRO
Already subscribed?
Connect
Ask anything
41 # Plain English: same include/exclude recursion as brute force, but
42 # remembers the answer for every (item, remaining capacity) pair it
43 # has already solved.
44 # ==
45 def knapsack_top_down(weights, values, capacity):
46 """Recursive solution with a cache. Time complexity: O(n * capacity)."""
47 n = len(weights)
48 memo = {}
49
50 def helper(i, remaining_capacity):
51 if i == 0 or remaining_capacity == 0:
52 return 0
53 if (i, remaining_capacity) in memo:
54 return memo[(i, remaining_capacity)]
55
56 if weights[i - 1] > remaining_capacity:
57 result = helper(i - 1, remaining_capacity)
58 else:
59 include = values[i - 1] + helper(i - 1, remaining_capacity - weights[i - 1])
60 exclude = helper(i - 1, remaining_capacity)
61 result = max(include, exclude)
62
63 memo[(i, remaining_capacity)] = result
64 return result
BLACKBOX Agent
Open Website
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.11 (64-bit)
Go Live
AI Code Chat
ENG
IN
11:39 AM
8/24/2026

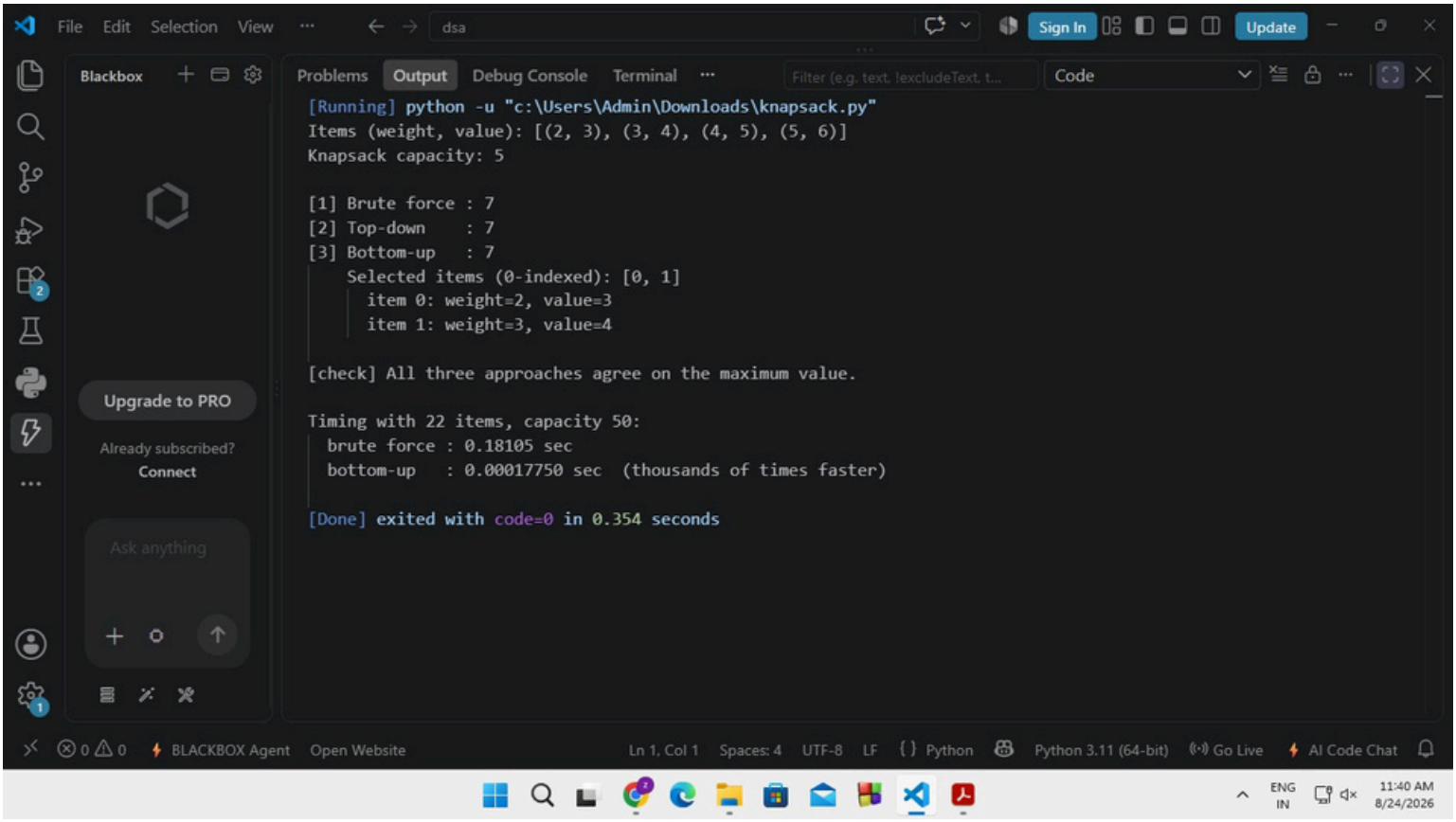

File Edit Selection View
dsa
Sign In
Update
Blackbox
nment 1.py
fibonacci (4).py
unit 2 assignment 2.py
fibonacci (1).py
fibonacci (3).py
knapsack.py C:\...\Downloads
C: > Users > Admin > Downloads > knapsack.py > ...
Upgrade to PRO
Already subscribed?
Connect
Ask anything
45 def knapsack_top_down(weights, values, capacity):
50 def helper(i, remaining_capacity):
63 memo[(i, remaining_capacity)] = result
64 return result
65
66 return helper(n, capacity)
67
68
69 # ==
70 # 3. BOTTOM-UP (tabulation) \u2014 the main, recommended solution
71 # Plain English: build a grid where rows are items and columns are
72 # "capacity used so far". Fill it from the top-left, one cell at a
73 # time, using only cells already filled.
74 # ==
75 def build_knapsack_table(weights, values, capacity):
76 """Fills an (n+1) x (capacity+1) grid where dp[i][w] is the best
77 value achievable using only the first i items, with a weight
78 limit of w."""
79 n = len(weights)
80 dp = [[0] * (capacity + 1) for _ in range(n + 1)]
81
82 for i in range(1, n + 1):
83 for w in range(capacity + 1):
84 if weights[i - 1] > w:
BLACKBOX Agent
Open Website
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.11 (64-bit)
Go Live
AI Code Chat
ENG
IN
11:39 AM
8/24/2026


File Edit Selection View
dsa
Sign In
Update
Blackbox
nment 1.py
fibonacci (4).py
unit 2 assignment 2.py
fibonacci (1).py
fibonacci (3).py
knapsack.py C:\...\Downloads
C: > Users > Admin > Downloads > knapsack.py > ...
Upgrade to PRO
Already subscribed?
Connect
Ask anything
def build_knapsack_table(weights, values, capacity):
for i in range(1, n + 1):
for w in range(capacity + 1):
if weights[i - 1] > w:
dp[i][w] = dp[i - 1][w] # can't fit -> skip it
else:
skip = dp[i - 1][w]
take = values[i - 1] + dp[i - 1][w - weights[i - 1]]
dp[i][w] = max(skip, take) # best of skipping or taking
return dp
def reconstruct_selected_items(dp, weights, values, capacity):
"""Walks BACKWARDS through the filled table to recover WHICH items
were actually chosen, not just the best total value."""
n = len(weights)
w = capacity
selected = []
for i in range(n, 0, -1):
if dp[i][w] != dp[i - 1][w]:
selected.append(i - 1) # item index (0-based) was included
w -= weights[i - 1]
selected.reverse()
return selected
BLACKBOX Agent
Open Website
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.11 (64-bit)
Go Live
AI Code Chat
ENG
IN
11:40 AM
8/24/2026


File Edit Selection View
dsa
Sign In
Update
Blackbox
nment 1.py
fibonacci (4).py
unit 2 assignment 2.py
fibonacci (1).py
fibonacci (3).py
knapsack.py C:\...\Downloads
C: > Users > Admin > Downloads > knapsack.py > ...
Upgrade to PRO
Already subscribed?
Connect
Ask anything
def reconstruct_selected_items(dp, weights, values, capacity):
w -= weights[i - 1]
selected.reverse()
return selected
def knapsack_bottom_up(weights, values, capacity):
"""Main entry point.
Returns: (max_value, selected_item_indices, dp_table)"""
dp = build_knapsack_table(weights, values, capacity)
n = len(weights)
max_value = dp[n][capacity]
selected = reconstruct_selected_items(dp, weights, values, capacity)
return max_value, selected, dp
==
4. DEMO / DRIVER CODE
==
if __name__ == "__main__":
weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
capacity = 5
BLACKBOX Agent
Open Website
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.11 (64-bit)
Go Live
AI Code Chat
ENG
IN
11:40 AM
8/24/2026


File Edit Selection View
dsa
Sign In
Update
Blackbox
nment 1.py
fibonacci (4).py
unit 2 assignment 2.py
fibonacci (1).py
fibonacci (3).py
knapsack.py C:\...\Downloads
C: > Users > Admin > Downloads > knapsack.py > ...
values = [3, 4, 5, 6]
capacity = 5
print("Items (weight, value):", list(zip(weights, values)))
print(f"Knapsack capacity: {capacity}\n")
print(f"[1] Brute force : {knapsack_brute_force(weights, values, capacity)}")
print(f"[2] Top-down : {knapsack_top_down(weights, values, capacity)}")
max_value, selected, dp = knapsack_bottom_up(weights, values, capacity)
print(f"[3] Bottom-up : {max_value}")
print(f" Selected items (0-indexed): {selected}")
for idx in selected:
print(f" item {idx}: weight={weights[idx]}, value={values[idx]}")
---- cross-check: brute force, top-down, and bottom-up all agree ----
assert knapsack_brute_force(weights, values, capacity) == max_value
assert knapsack_top_down(weights, values, capacity) == max_value
print("\n[check] All three approaches agree on the maximum value.")
---- timing comparison on a moderately large input ----------
import random
random.seed(42)
n_big = 22
Upgrade to PRO
Already subscribed?
Connect
Ask anything
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.11 (64-bit)
Go Live
AI Code Chat
BLACKBOX Agent
Open Website
ENG
IN
11:40 AM
8/24/2026


File Edit Selection View
dsa
Sign In
Update
Blackbox
nment 1.py
fibonacci (4).py
unit 2 assignment 2.py
fibonacci (1).py
fibonacci (3).py
knapsack.py C:\...\Downloads
C: > Users > Admin > Downloads > knapsack.py > ...
assert knapsack_top_down(weights, values, capacity) == max_value
print("\n[check] All three approaches agree on the maximum value.")
---- timing comparison on a moderately large input ----------
import random
random.seed(42)
n_big = 22
big_weights = [random.randint(1, 15) for _ in range(n_big)]
big_values = [random.randint(1, 20) for _ in range(n_big)]
big_capacity = 50
t_brute = timeit.timeit(lambda: knapsack_brute_force(big_weights, big_values, big_capacity),
t_bottom_up = timeit.timeit(lambda: knapsack_bottom_up(big_weights, big_values, big_capacity
print(f"\nTiming with {n_big} items, capacity {big_capacity}:")
print(f" brute force : {t_brute:.5f} sec")
print(f" bottom-up : {t_bottom_up:.8f} sec (thousands of times faster)")
Upgrade to PRO
Already subscribed?
Connect
Ask anything
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.11 (64-bit)
Go Live
AI Code Chat
BLACKBOX Agent
Open Website
ENG
IN
11:40 AM
8/24/2026


File Edit Selection View
dsa
Sign In
Update
Blackbox
Problems
Output
Debug Console
Terminal
Filter (e.g. text, !excludeText, t...
Code
[Running] python -u "c:\Users\Admin\Downloads\knapsack.py"
Items (weight, value): [(2, 3), (3, 4), (4, 5), (5, 6)]
Knapsack capacity: 5
[1] Brute force : 7
[2] Top-down : 7
[3] Bottom-up : 7
Selected items (0-indexed): [0, 1]
item 0: weight=2, value=3
item 1: weight=3, value=4
[check] All three approaches agree on the maximum value.
Timing with 22 items, capacity 50:
brute force : 0.18105 sec
bottom-up : 0.00017750 sec (thousands of times faster)
[Done] exited with code=0 in 0.354 seconds
Upgrade to PRO
Already subscribed?
Connect
Ask anything
0 0
BLACKBOX Agent
Open Website
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.11 (64-bit)
Go Live
AI Code Chat
ENG
IN
11:40 AM
8/24/2026